Failure (prove the suite catches a regression) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

This exercise is a **test of the tests** — like pressing the test button on a smoke alarm, instead of just trusting it works.

The process: record your suite's current score as a baseline. Pick one instruction in the prompt that matters (here: "always mention the 30-day return policy"). Delete that one instruction. Re-run the exact same suite, unchanged. Compare the new score to the baseline.

The exact pieces:

- **Keep the prompt in a file** (`practice/support_prompt.txt`), and read it fresh on every model call — then you can edit the file while the script waits, and the next run uses the new version.
- **A small, fixed task list** — 4 customer questions about returns, refunds and exchanges, each with the phrase a correct reply must contain (`"30"` , for the 30-day policy).
- **`run_eval_suite()`** returns `{task_id: True/False}` — one pass/fail per task, so you can see *which* task changed, not just the total.
- **`input("...press Enter")`** — pauses the script while you edit and save the prompt file.
- **Change exactly one thing.** If you change several things and the score drops, you won't know which change the suite caught.
- **Newly failed tasks** — the tasks that passed before and fail after. A `for` loop over the baseline results finds them.

Before you run anything, write down your *prediction*: which tasks should fail, and why?

Hint 2 — The plan, and almost the whole thing

```
same TASKS and run_eval_suite()

baseline = run_eval_suite()
print baseline passed / total
input("remove the 30-day line, save, press Enter")
after = run_eval_suite()

newly_failed = []
for each task_id in baseline:
    if it passed in baseline and failed after: add it to newly_failed

if newly_failed: the suite caught it — print which tasks
else: the suite has a hole — tighten the rule that should have caught it
```

Here's the comparison — fill in the loop yourself:

```
# practice/regression_catch_practice.py
def find_newly_failed(baseline, after):
    newly_failed = []
    # your turn: loop over baseline; append task_id when
    # baseline[task_id] is True and after[task_id] is False
    ...
    return newly_failed
```

Full solution: Show me the solution